고객을 세그먼테이션하자 [프로젝트] - 신정아

11-2. 데이터 불러오기

데이터 살펴보기

- **테이블에 있는 10개의 행만 출력하기**

```
SELECT *
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
LIMIT 10
```

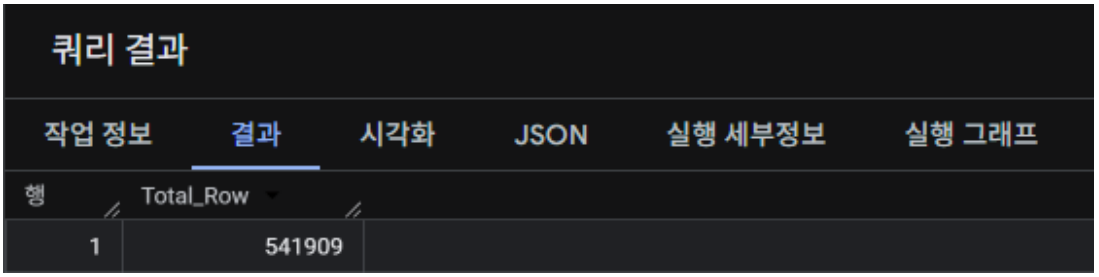
[결과 이미지]

행	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
1	536365	85123A	WHITE HANGING HEART T-LIG...	6	2010-12-01 08:26:00 UTC	2.55	17850	United Kingdom
2	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom
3	536365	84406B	CREAM CUPID HEARTS COAT H...	8	2010-12-01 08:26:00 UTC	2.75	17850	United Kingdom
4	536365	84029G	KNITTED UNION FLAG HOT WA...	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom
5	536365	84029E	RED WOOLLY HOTTIE WHITE H...	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom
6	536365	22752	SET 7 BABUSHKA NESTING BO...	2	2010-12-01 08:26:00 UTC	7.65	17850	United Kingdom
7	536365	21730	GLASS STAR FROSTED T-LIGHT...	6	2010-12-01 08:26:00 UTC	4.25	17850	United Kingdom
8	536366	22633	HAND WARMER UNION JACK	6	2010-12-01 08:28:00 UTC	1.85	17850	United Kingdom
9	536366	22632	HAND WARMER RED POLKA DOT	6	2010-12-01 08:28:00 UTC	1.85	17850	United Kingdom
10	536367	84879	ASSORTED COLOUR BIRD ORN...	32	2010-12-01 08:34:00 UTC	1.69	13047	United Kingdom

- **전체 데이터는 몇 행으로 구성되어 있는지 확인하기**

```
SELECT COUNT(*) AS Total_Row
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
```

[결과 이미지]

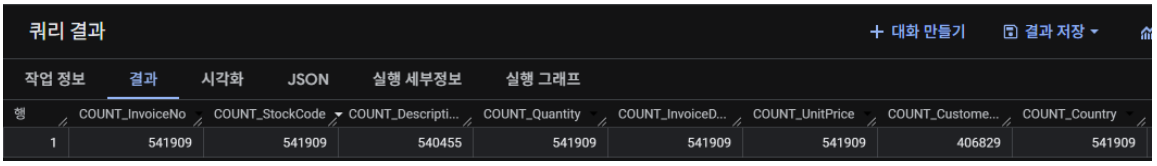


데이터 수 세기

- **COUNT 함수를 사용해서, 각 컬럼별 데이터 포인트의 수를 세어 보기**

```
SELECT COUNT(InvoiceNo) AS COUNT_InvoiceNo,
       COUNT(StockCode) AS COUNT_StockCode,
       COUNT(Description) AS COUNT_Description,
       COUNT(Quantity) AS COUNT_Quantity,
       COUNT(InvoiceDate) AS COUNT_InvoiceDate,
       COUNT(UnitPrice) AS COUNT_UnitPrice,
       COUNT(CustomerID) AS COUNT_CustomerID,
       COUNT(Country) AS COUNT_Country
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.da
ta`
```

[결과 이미지]



11-4. 데이터 전처리 방법(1): 결측치 제거

컬럼 별 누락된 값의 비율 계산

- **각 컬럼 별 누락된 값의 비율을 계산**
  - **방법1) 각 컬럼에 대해서 누락 값을 계산한 후, 계산된 누락 값을 UNION ALL을 통해 합치기**

1) 각 컬럼에 대한 누락값 계산 : SELECT 컬럼명, 결측비율계산 FROM 테이블명
2) 1)에서 도출한 테이블 컬럼명 column_name, missing_percentage 의 테이블을 UNION ALL 을 이용하여 세로로 합치기
※ UNION vs UNION ALL :
- UNION : 테이블을 아래로, 세로로 붙이는 형태 / 컬럼명이 달라도 적용 가능 (중복 제거)
- UNION ALL : 중복 포함하여 테이블을 세로로 합칠 때 사용

```
SELECT
    'InvoiceNo' AS column_name,
    ROUND(SUM(CASE WHEN InvoiceNo IS NULL THEN 1 ELSE 0
END) / COUNT(*) * 100, 2) AS missing_percentage
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.da
ta`
UNION ALL
SELECT
    'StockCode' AS column_name,
    ROUND(SUM(CASE WHEN StockCode IS NULL THEN 1 ELSE 0
END) / COUNT(*) * 100, 2) AS missing_percentage
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.da
ta`
UNION ALL
SELECT
    'Description' AS column_name,
    ROUND(SUM(CASE WHEN Description IS NULL THEN 1 ELSE
0 END) / COUNT(*) * 100, 2) AS missing_percentage
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.da
ta`
UNION ALL
SELECT
    'Quantity' AS column_name,
    ROUND(SUM(CASE WHEN Quantity IS NULL THEN 1 ELSE 0 E
ND) / COUNT(*) * 100, 2) AS missing_percentage
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.da
ta`
UNION ALL
SELECT
    'InvoiceDate' AS column_name,
```

```
    ROUND(SUM(CASE WHEN InvoiceDate IS NULL THEN 1 ELSE
0 END) / COUNT(*) * 100, 2) AS missing_percentage
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.da
ta`
UNION ALL
SELECT
    'UnitPrice' AS column_name,
    ROUND(SUM(CASE WHEN UnitPrice IS NULL THEN 1 ELSE 0
END) / COUNT(*) * 100, 2) AS missing_percentage
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.da
ta`
UNION ALL
SELECT
    'CustomerID' AS column_name,
    ROUND(SUM(CASE WHEN CustomerID IS NULL THEN 1 ELSE 0
END) / COUNT(*) * 100, 2) AS missing_percentage
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.da
ta`
UNION ALL
SELECT
    'Country' AS column_name,
    ROUND(SUM(CASE WHEN Country IS NULL THEN 1 ELSE 0 EN
D) / COUNT(*) * 100, 2) AS missing_percentage
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.da
ta`
```

[결과 이미지 1]

쿼리 결과

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

행	column_name	missing_percenta...
1	InvoiceNo	0.0
2	StockCode	0.0
3	Description	0.27
4	Quantity	0.0
5	InvoiceDate	0.0
6	UnitPrice	0.0
7	CustomerID	24.93
8	Country	0.0

- **방법2) 각 컬럼에 대해서 누락 값을 계산한 후, 계산된 누락 값을 UNION ALL을 통해 합쳐서 새로운 테이블 생성, 이후 새로운 테이블에서 컬럼명, 결측률을 계산하여 추출**
  1) 각 컬럼에 대한 누락값 계산 : SELECT 컬럼명, COUNT(컬럼명), COUNT(*)
  2) 1)에서 도출한 테이블 컬럼명 column_name, 해당 컬럼 데이터수, 전체 행수로 구성된 테이블을 UNION ALL 을 이용하여 세로로 합친 후, 새로운 테이블 생성
  3) SELECT와 산술연산을 이용하여 결측율 계산,
  4) FROM 뒤 테이블명은 앞선 (2) 단계에서 생성한 쿼리 이용

```
SELECT column_name, ROUND((total - column_value) / total
* 100, 2)
FROM (
     SELECT 'InvoiceNo' AS column_name, COUNT(InvoiceNo)
AS column_value, COUNT(*) AS total
     FROM `project-5faeab26-b699-4f50-a43.modulabs_proje
ct.data`
     UNION ALL
     SELECT 'StockCode' AS column_name, COUNT(StockCode)
AS column_value, COUNT(*) AS total
     FROM `project-5faeab26-b699-4f50-a43.modulabs_proje
ct.data`
     UNION ALL
     SELECT 'Description' AS column_name, COUNT(Descript
ion) AS column_value, COUNT(*) AS total
```

```
     FROM `project-5faeab26-b699-4f50-a43.modulabs_proje
ct.data`
     UNION ALL
     SELECT 'Quantity' AS column_name, COUNT(Quantity) A
S column_value, COUNT(*) AS total
     FROM `project-5faeab26-b699-4f50-a43.modulabs_proje
ct.data`
     UNION ALL
     SELECT 'InvoiceDate' AS column_name, COUNT(InvoiceD
ate) AS column_value, COUNT(*) AS total
     FROM `project-5faeab26-b699-4f50-a43.modulabs_proje
ct.data`
     UNION ALL
     SELECT 'UnitPrice' AS column_name, COUNT(UnitPrice)
AS column_value, COUNT(*) AS total
     FROM `project-5faeab26-b699-4f50-a43.modulabs_proje
ct.data`
     UNION ALL
     SELECT 'CustomerID' AS column_name, COUNT(CustomerI
D) AS column_value, COUNT(*) AS total
     FROM `project-5faeab26-b699-4f50-a43.modulabs_proje
ct.data`
     UNION ALL
     SELECT 'Country' AS column_name, COUNT(Country) AS
column_value, COUNT(*) AS total
     FROM `project-5faeab26-b699-4f50-a43.modulabs_proje
ct.data`)
     AS column_data
```

결측치 처리 전략

- `StockCode = '85123A'`의 `Description`을 추출하는 쿼리문을 작성하기

```
SELECT DISTINCT Description
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.da
```

```
ta`
WHERE StockCode = '85123A'
```

[결과 이미지]

행	Description
1	WHITE HANGING HEART T-LIGHT HOLDER
2	?
3	wrongly marked carton 22804
4	CREAM HANGING HEART T-LIGHT HOLDER

- **`StockCode = '85123A'` 의 `Description` 을 추출(해당 데이터의 고유값 및 고유값 개수 확인) 쿼리문**

```
SELECT Description, COUNT(*) AS cnt
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.da
ta`
WHERE StockCode = '85123A'
GROUP BY 1
ORDER BY cnt DESC
```

[결과 이미지]

행	Description	cnt
1	WHITE HANGING HEART T-LIGHT HOLDER	2302
2	CREAM HANGING HEART T-LIGHT HOLDER	9
3	wrongly marked carton 22804	1
4	?	1

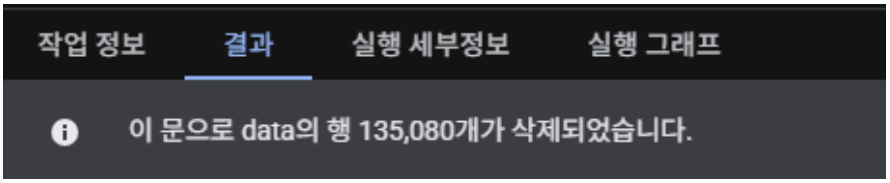
결측치 처리

- **DELETE 구문을 사용하며, WHERE 절을 통해 데이터를 제거할 조건을 제시**

```
# DELETE 구문 사용 전, [SELECT * FROM 테이블명 WHERE 조건]으
로
# 삭제할 데이터를 확인 후, DELETE FROM 테이블명 WHERE 조건 적
용
DELETE FROM `project-5faeab26-b699-4f50-a43.modulabs_pro
```

```
ject.data`
WHERE Description IS NULL OR CustomerID IS NULL
```

[결과 이미지]



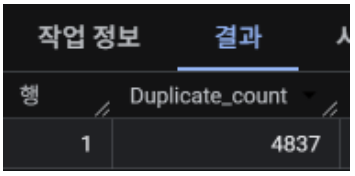
11-5. 데이터 전처리(2): 중복값 처리

중복값 확인

- **중복된 행의 수를 세어보기**
  - **8개의 컬럼에 그룹 함수를 적용한 후, COUNT가 1보다 큰 데이터를 세어보기**

```
SELECT COUNT(*) AS Duplicate_count
FROM(
SELECT *,
       COUNT(*) AS cnt
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.da
ta`
GROUP BY InvoiceNo, StockCode, Description, Quantity, In
voiceDate, UnitPrice, CustomerID, Country
HAVING cnt > 1)
```

[결과 이미지 - 중복된 행 수 계산]



  - **8개의 컬럼에 그룹 함수를 적용한 후, COUNT가 1보다 큰 데이터 확인**

```
SELECT InvoiceNo, StockCode, Description, Quantity, Invo
iceDate, UnitPrice, CustomerID, Country,
```

```
        COUNT(*) AS cnt
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
GROUP BY 1, 2, 3, 4, 5, 6, 7, 8
HAVING cnt > 1
```

[결과 이미지 - FROM 속 테이블]

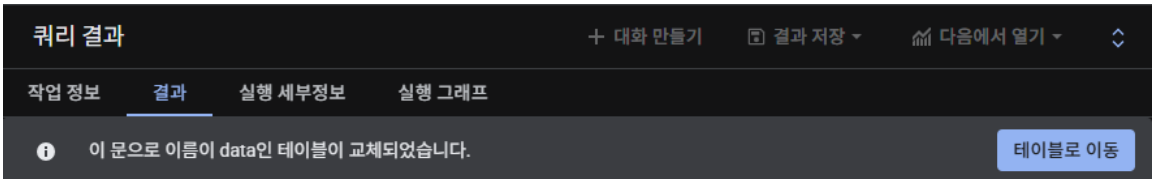
행	InvoiceNo	StockCode	Description	Quantity	InvoiceDate
1	576607	22940	FELTCRAFT CHRISTMAS FAIRY	8	2011-11-15 15
2	553731	23208	LUNCH BAG VINTAGE LEAF DE...	10	2011-05-19 08
3	555383	21828	EIGHT PIECE SNAKE SET	2	2011-06-02 15
4	559665	85061W	WHITE JEWELLED HEART DEC...	1	2011-07-11 13
5	537140	22632	HAND WARMER RED RETROSP...	1	2010-12-05 12
6	537155	21786	POLKADOT RAIN HAT	1	2010-12-05 13

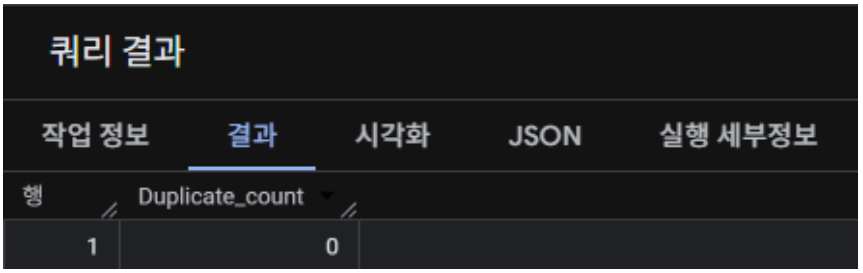
중복값 처리

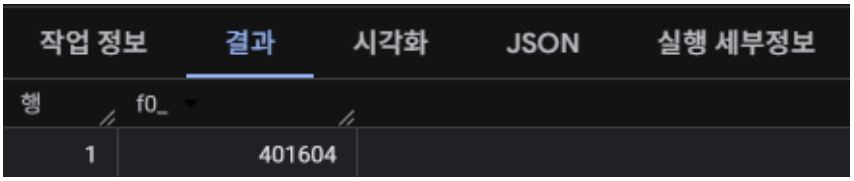
- **중복값을 제거하는 쿼리문 작성하기**
  - **CREATE OR REPLACE TABLE 구문을 활용하여 모든 컬럼(*)을 DISTINCT 한 데이터로 업데이트**

```
CREATE OR REPLACE TABLE `project-5faeab26-b699-4f50-a43.modulabs_project.data`
AS
SELECT DISTINCT *
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
```

[결과 이미지]







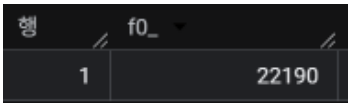
11-6. 데이터 전처리(3): 오류값 처리

`InvoiceNo` 살펴보기

- **고유(unique)한 `InvoiceNo`의 개수를 출력하기**

```
SELECT COUNT(*)
FROM (
SELECT InvoiceNo, COUNT(*) AS cnt
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
GROUP BY 1
ORDER BY cnt DESC)
```

[결과 이미지]



- **고유한 `InvoiceNo`를 앞에서부터 100개를 출력하기**

```
SELECT DISTINCT InvoiceNo
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
LIMIT 100
```

[결과 이미지]

행	InvoiceNo
1	541431
2	C541433
3	537626
4	542237
5	549222
6	556201
7	562032
8	573511
9	581180
10	539318
11	541998
12	548955

- **`InvoiceNo` 가 'C'로 시작하는 행을 필터링 할 수 있는 쿼리문을 작성하기 (100행까지만 출력)**

```
SELECT *
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
WHERE InvoiceNo LIKE 'C%'
LIMIT 100
```

[결과 이미지]

행	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
1	C541433	23166	MEDIUM CERAMIC TOP STORA...	-74215	2011-01-18 10:17:00 UTC	1.04	12346	United Kingdom
2	C545329	M	Manual	-1	2011-03-01 15:47:00 UTC	183.75	12352	Norway
3	C545329	M	Manual	-1	2011-03-01 15:47:00 UTC	280.05	12352	Norway
4	C545330	M	Manual	-1	2011-03-01 15:49:00 UTC	376.5	12352	Norway
5	C547388	22784	LANTERN CREAM GAZEBO	-3	2011-03-22 16:07:00 UTC	4.95	12352	Norway
6	C547388	84050	PINK HEART SHAPE EGG FRYIN...	-12	2011-03-22 16:07:00 UTC	1.65	12352	Norway
7	C547388	22413	METAL SIGN TAKE IT OR LEAVE...	-6	2011-03-22 16:07:00 UTC	2.95	12352	Norway
8	C547388	22701	PINK DOG BOWL	-6	2011-03-22 16:07:00 UTC	2.95	12352	Norway
9	C547388	22645	CERAMIC HEART FAIRY CAKE ...	-12	2011-03-22 16:07:00 UTC	1.45	12352	Norway

- **구매 건 상태가 `Canceled` 인 데이터의 비율(%) - 소수점 첫번째 자리까지**

```
SELECT ROUND(SUM(CASE WHEN InvoiceNo LIKE 'C%' THEN 1 ELSE 0 END)/COUNT(*)*100,1)
FROM `project-5faeab26-b699-4f50-a43.modulabs_project.data`
```

[결과 이미지]

행	f0_
1	2.2

`StockCode` 살펴보기

- **고유한 `StockCode`의 개수를 출력하기**

```
# [[YOUR QUERY]]
```

[결과 이미지를 넣어주세요]

- **어떤 제품이 가장 많이 판매되었는지 보기 위하여 `StockCode` 별 등장 빈도를 출력하기**
  - 상위 10개의 제품들을 출력하기

```
SELECT StockCode, COUNT(*) AS sell_cnt
FROM project_name.modulabs_project.data
# [[YOUR QUERY]]
ORDER BY sell_cnt DESC
# [[YOUR QUERY]];
```

[결과 이미지를 넣어주세요]

- **`StockCode`의 컬럼에 있던 값 중에서 숫자를 제외한 문자만 남기고 문자가 몇 자리 수 인지 세고**
  - **숫자가 0~1개인 값**들에는 어떤 코드들이 들어가 있는지 출력하기

```
SELECT DISTINCT StockCode, number_count
FROM (
  SELECT StockCode,
    LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode,
r'[0-9]', '')) AS number_count
```

```
  FROM project_name.modulabs_project.data
)
WHERE # [[YOUR QUERY]];
```

[결과 이미지를 넣어주세요]

- `StockCode` **의 컬럼에 있던 값 중에서 숫자를 제외한 문자만 남기고 문자가 몇 자리 수 인지 세고**
  - **숫자가 0~1개인 값들을 가지고 있는 데이터 수는 전체 데이터 수 대비 몇 퍼센트**인지 구하기 (소수점 두 번째 자리까지)

```
SELECT DISTINCT StockCode, number_count
FROM (
  SELECT StockCode,
    LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode,
r'[0-9]', '')) AS number_count
  FROM project_name.modulabs_project.data
)
WHERE # [[YOUR QUERY]];
```

[결과 이미지를 넣어주세요]

- **제품과 관련되지 않은 거래 기록을 제거하기**

```
DELETE FROM project_name.modulabs_project.data
WHERE StockCode IN (
  SELECT DISTINCT StockCode
  FROM (
    # [[YOUR QUERY]]
);
```

[결과 이미지를 넣어주세요]

`Description` 살펴보기

- **고유한 Description 별 출현 빈도를 계산하고 상위 30개를 출력하기**

```
SELECT Description, COUNT(*) AS description_cnt
FROM project_name.modulabs_project.data
# [[YOUR QUERY]]
```

[결과 이미지를 넣어주세요]

- **서비스 관련 정보를 포함하는 행들을 제거하기**

```
DELETE
FROM project_name.modulabs_project.data
WHERE
# [[YOUR QUERY]]
```

[결과 이미지를 넣어주세요]

- **대소문자를 혼합하고 있는 데이터를 대문자로 표준화 하기**

```
CREATE OR REPLACE TABLE project_name.modulabs_project.da
ta AS
SELECT
  * EXCEPT (Description),
  # [[YOUR QUERY]] AS Description
FROM project_name.modulabs_project.data;
```

[결과 이미지를 넣어주세요]

`UnitPrice` 살펴보기

- `UnitPrice`의 최솟값, 최댓값, 평균을 구하기

```
SELECT # [[YOUR QUERY]] AS min_price, # [[YOUR QUERY]] A
S max_price, # [[YOUR QUERY]] AS avg_price
FROM project_name.modulabs_project.data;
```

[결과 이미지를 넣어주세요]

- 단가가 0원인 거래의 개수, 구매 수량(`Quantity`)의 최솟값, 최댓값, 평균 구하기

```
SELECT # [[YOUR QUERY]] AS cnt_quantity, # [[YOUR QUER
Y]] AS min_quantity, # [[YOUR QUERY]] AS max_quantity, #
[[YOUR QUERY]] AS avg_quantity
FROM project_name.modulabs_project.data
WHERE # [[YOUR QUERY]];
```

[결과 이미지를 넣어주세요]

- `UnitPrice = 0`를 제거하고 일관된 데이터셋을 유지하기

```
CREATE OR REPLACE TABLE project_name.modulabs_project.da
ta AS
SELECT *
FROM project_name.modulabs_project.data
WHERE # [[YOUR QUERY]];
```

[결과 이미지를 넣어주세요]

11-7. RFM 스코어

Recency

- `InvoiceDate` **컬럼을 연월일 자료형으로 변경하기**

```
SELECT # [[YOUR QUERY]] AS InvoiceDay, *
FROM project_name.modulabs_project.data;
```

[결과 이미지를 넣어주세요]

- **가장 최근 구매 일자를 MAX() 함수로 찾아보기**

```
SELECT
   # [[YOUR QUERY]] AS most_recent_date,
   # [[YOUR QUERY]] AS InvoiceDay,
   *
FROM project_name.modulabs_project.data;
```

[결과 이미지를 넣어주세요]

- **유저 별로 가장 큰 InvoiceDay를 찾아서 가장 최근 구매일로 저장하기**

```
SELECT
    CustomerID,
    # [[YOUR QUERY]] AS InvoiceDay
FROM project_name.modulabs_project.data
# [[YOUR QUERY]];
```

[결과 이미지를 넣어주세요]

- **가장 최근 일자(`most_recent_date`)와 유저별 마지막 구매일(`InvoiceDay`)간의 차이를 계산하기**

```
SELECT
  CustomerID,
  EXTRACT(DAY FROM MAX(InvoiceDay) OVER () - InvoiceDay)
AS recency
FROM (
  SELECT
    CustomerID,
    MAX(DATE(InvoiceDate)) AS InvoiceDay
  FROM project_name.modulabs_project.data
  GROUP BY CustomerID
);
```

[결과 이미지를 넣어주세요]

- **최종 데이터 셋에 필요한 데이터들을 각각 정제해서 이어붙이고 지금까지의 결과를 `user_r` 이라는 이름의 테이블로 저장하기**

```
CREATE OR REPLACE TABLE project_name.modulabs_project.us
er_r AS
# [[YOUR QUERY]]
```

[결과 이미지를 넣어주세요]

Frequency

- **고객마다 고유한 InvoiceNo의 수를 세어보기**

```
SELECT
  CustomerID,
  # [[YOUR QUERY]] AS purchase_cnt
FROM project_name.modulabs_project.data
# [[YOUR QUERY]];
```

[결과 이미지를 넣어주세요]

- **각 고객 별로 구매한 아이템의 총 수량 더하기**

```
SELECT
  CustomerID,
  # [[YOUR QUERY]] AS item_cnt
FROM project_name.modulabs_project.data
# [[YOUR QUERY]];
```

[결과 이미지를 넣어주세요]

- **전체 거래 건수 계산와 구매한 아이템의 총 수량 계산의 결과를 합쳐서 `user_rf` 라는 이름의 테이블에 저장하기**

```
CREATE OR REPLACE TABLE project_name.modulabs_project.us
er_rf AS

-- (1) 전체 거래 건수 계산
WITH purchase_cnt AS (
  # [[YOUR QUERY]]
),

-- (2) 구매한 아이템 총 수량 계산
item_cnt AS (
  # [[YOUR QUERY]]
)

-- 기존의 user_r에 (1)과 (2)를 통합
SELECT
  pc.CustomerID,
  pc.purchase_cnt,
  ic.item_cnt,
  ur.recency
FROM purchase_cnt AS pc
JOIN item_cnt AS ic
```

```
  ON pc.CustomerID = ic.CustomerID
JOIN project_name.modulabs_project.user_r AS ur
  ON pc.CustomerID = ur.CustomerID;
```

[결과 이미지를 넣어주세요]

Monetary

- **고객별 총 지출액 계산 (소수점 첫째 자리에서 반올림)**

```
SELECT
  CustomerID,
  # [[YOUR QUERY]] AS user_total
FROM project_name.modulabs_project.data
# [[YOUR QUERY]];
```

[결과 이미지를 넣어주세요]

- **고객별 평균 거래 금액 계산**
  - **고객별 평균 거래 금액을 구하기 위해 1) `data` 테이블을 `user_rf` 테이블과 조인 (LEFT JOIN) 한 후, 2) `purchase_cnt` 로 나누어서 3) `user_rfm` 테이블로 저장하기**

```
CREATE OR REPLACE TABLE project_name.modulabs_project.us
er_rfm AS
SELECT
  rf.CustomerID AS CustomerID,
  rf.purchase_cnt,
  rf.item_cnt,
  rf.recency,
  ut.user_total,
  # [[YOUR QUERY]] AS user_average
FROM project_name.modulabs_project.user_rf rf
LEFT JOIN (
```

```
  -- 고객 별 총 지출액
  SELECT
    # [[YOUR QUERY]]
) ut
ON rf.CustomerID = ut.CustomerID;
```

[결과 이미지를 넣어주세요]

RFM 통합 테이블 출력하기

- **최종 user_rfm 테이블을 출력하기**

```
# [[YOUR QUERY]];
```

[결과 이미지를 넣어주세요]

11-8. 추가 Feature 추출

1. 구매하는 제품의 다양성

- **1) 고객 별로 구매한 상품들의 고유한 수를 계산하기**
  2) `user_rfm` 테이블과 결과를 합치기
  3) `user_data` 라는 이름의 테이블에 저장하기

```
CREATE OR REPLACE TABLE project_name.modulabs_project.user_
data AS
WITH unique_products AS (
  SELECT
    CustomerID,
    COUNT(DISTINCT StockCode) AS unique_products
  FROM project_name.modulabs_project.data
  GROUP BY CustomerID
```

```
)
SELECT ur.*, up.* EXCEPT (CustomerID)
FROM project_name.modulabs_project.user_rfm AS ur
JOIN unique_products AS up
ON ur.CustomerID = up.CustomerID;
```

[결과 이미지를 넣어주세요]

2. 평균 구매 주기

- **고객들의 쇼핑 패턴을 이해하는 것을 목표 (고객 별 재방문 주기 살펴보기)**
  - **균 구매 소요 일수를 계산하고, 그 결과를 `user_data`에 통합**

```
CREATE OR REPLACE TABLE project_name.modulabs_project.user_
data AS
WITH purchase_intervals AS (
  -- (2) 고객 별 구매와 구매 사이의 평균 소요 일수
  SELECT
    CustomerID,
    CASE WHEN ROUND(AVG(interval_), 2) IS NULL THEN 0 ELSE
ROUND(AVG(interval_), 2) END AS average_interval
  FROM (
    -- (1) 구매와 구매 사이에 소요된 일수
    SELECT
      CustomerID,
      DATE_DIFF(InvoiceDate, LAG(InvoiceDate) OVER (PARTITI
ON BY CustomerID ORDER BY InvoiceDate), DAY) AS interval_
    FROM
      project_name.modulabs_project.data
    WHERE CustomerID IS NOT NULL
  )
  GROUP BY CustomerID
)

SELECT u.*, pi.* EXCEPT (CustomerID)
```

```
FROM project_name.modulabs_project.user_data AS u
LEFT JOIN purchase_intervals AS pi
ON u.CustomerID = pi.CustomerID;
```

[결과 이미지를 넣어주세요]

3. 구매 취소 경향성

- **고객의 취소 패턴 파악하기**
  1) 취소 빈도(cancel_frequency) : 고객 별로 취소한 거래의 총 횟수
  2) 취소 비율(cancel_rate) : 각 고객이 한 모든 거래 중에서 취소를 한 거래의 비율
    - **취소 빈도와 취소 비율을 계산하고 그 결과를 `user_data`에 통합하기**
      (취소 비율은 소수점 두번째 자리)

```
CREATE OR REPLACE TABLE project_name.modulabs_project.user_
data AS

WITH TransactionInfo AS (
  SELECT
    CustomerID,
    # [[YOUR QUERY]] AS total_transactions,
    # [[YOUR QUERY]] AS cancel_frequency
  FROM project_name.modulabs_project.data
  # [[YOUR QUERY]]
)

SELECT u.*, t.* EXCEPT(CustomerID), # [[YOUR QUERY]] AS can
cel_rate
FROM `project_name.modulabs_project.user_data` AS u
LEFT JOIN TransactionInfo AS t
ON # [[YOUR QUERY]];
```

[결과 이미지를 넣어주세요]

- **다양한 컬럼들을 활용하여 고객의 구매 패턴과 선호도를 보다 심층적으로 이해할 수 있도록 최종적으로 `user_data` 를 출력하기**

```
# [[YOUR QUERY]];
```

[결과 이미지를 넣어주세요]

회고

[회고 내용을 작성해주세요]

Keep :

Problem :

Try :